Presenter script

QIR Optimizer · BCSE307 Review 3 · about 20 minutes
Speaker 1: scenes 1–5 and 16 · Speaker 2: 6–9 · Speaker 3: 10–12 · Speaker 4: 13–15 and 17

SPEAKER 1

Scene 1: A self-verifying optimizer for quantum programs

Good morning. Our project is a compiler for quantum programs, and its main job is to make them smaller without changing what they do.

It does three things. It compiles quantum programs into QIR, the industry-standard intermediate representation built on LLVM. It removes every gate that provably does nothing. And it proves every single change is correct before keeping it.

This small program has five gates and an if-statement. Watch it shrink. At each step you can see the rule used and a tag saying the step was validated. We end with two gates, no branch, and a proof that the program behaves exactly the same.

The rest of this presentation explains how that happens.

Scene 2: Quantum computers make mistakes

First, why this matters. Quantum hardware today is noisy. Every gate has a small chance of going wrong, about 0.2 percent for a one-qubit gate and 1.5 percent for a two-qubit gate.

Those small chances multiply. With 20 gates, most runs are clean. With 300 gates, almost none are.

So every gate a compiler removes is free accuracy, with no new hardware needed. One of our benchmark programs, a 30-gate adder, goes from a 39 percent chance of an error-free run to 94 percent after optimization.

Scene 3: A qubit in one minute

We only need a few quantum ideas.

A qubit is like a bit: it can be 0 or 1. Unlike a bit, it can also be in a mixture of both, called a superposition.

Both qubits start at 0, so every run measures 00.

H puts q0 into an even mixture. Measuring it now gives 0 half the time and 1 half the time. And measuring ends the mixture: you always get a plain 0 or 1.

CX links the two qubits. Now we only ever see 00 or 11, never 01 or 10. The qubits are entangled: neither one has a value of its own.

For our optimizer, the only question that matters is this: is a qubit definitely 0, definitely 1, or unknown? Keep that in mind; it comes back in the second optimization.

Scene 4: Gates: the instructions

Quantum programs are drawn as circuits. Each horizontal line is one qubit, and time runs from left to right. Each box or dot is a gate, which is an instruction.

X is NOT: it flips 0 and 1. H makes the 50/50 mixture we just saw. Z, S and T change something called phase, which is invisible on its own but matters in combination. The rotations turn a qubit by an angle, and two rotations on the same axis add up.

CX, the controlled NOT, flips the target, drawn as a circle with a plus, only when the control, the dot, is 1. CCX is the same with two controls, like an AND gate. SWAP exchanges two qubits. And Measure reads out the answer.

Now look at the last line of each entry: almost every gate undoes itself. Apply X twice and nothing happens. That one fact is what most of our optimizations exploit.

Scene 5: Some gates undo each other

Here is the simplest case. H twice, X twice, CX twice: three pairs that each undo themselves.

The optimizer finds each pair and removes it. At the end there are no gates left, and the tag says the result is proven equivalent: the measurements give exactly the same answers as before.

This is roughly what our Review 2 prototype did, by matching lines of text. The rest of the project is about doing it safely, on a real compiler IR, and going much further than adjacent pairs.

I'll now hand over to , who will show how quantum programs look inside a real compiler.

SPEAKER 2

Scene 6: QIR: quantum programs written as LLVM IR

Inside a compiler, the program is not a picture. It is QIR, the Quantum Intermediate Representation. QIR is defined by the QIR Alliance and is used by companies such as Microsoft, Quantinuum and NVIDIA.

The key point is that QIR is ordinary LLVM IR, the same IR that Clang and Rust compile to. Four lines show how it works.

A gate is just a function call.

A measurement writes its result into a result slot.

That result is read back as an ordinary one-bit value.

And the program branches on it, with a normal if-statement.

So a QIR program is quantum code and classical code mixed in one control-flow graph. That mix is exactly what our optimizer takes advantage of.

Scene 7: Nobody was optimizing both halves

Existing tools each see only half of that picture.

LLVM's optimizer sees the branches and the classical code. But to LLVM, a gate is a call to a function it knows nothing about, so it has to keep every gate.

Quantum circuit tools such as Qiskit and tket understand the gates, so they know H twice is nothing. But they work on circuits and barely see branches or classical code.

Our optimizer sits in the gap: it understands the gates, it sees the branches, and it proves every change.

Scene 8: What we built

This is the whole system, running live on the small program from the start. The circuit is at the top, and beside it is what the program does: set q0 to 1, copy it onto q1, apply H twice, and branch on a measurement.

There are three parts. The front-end compiles OpenQASM, a standard quantum programming language, into QIR. The optimizer turns QIR into smaller QIR. The verifier is a simulator that proves each change.

First the lexer turns characters into tokens. The parser builds a syntax tree. Semantic analysis checks names, sizes and indices. Then we generate QIR.

The optimizer reads that QIR back and builds a dependence graph, which shows which operations touch the same qubit. Then it optimizes, and the circuit at the top shrinks from five gates to two. Finally the simulator confirms that both versions produce exactly the same results.

Every one of these stages is our own code, running in this page.

> **If asked "why not a C++ LLVM pass, as in Review 2?":**

We wanted to build every compiler phase ourselves, including the front-end, and to run the real engine in the presentation. LLVM is still used: our tests run every output through LLVM's own verifier.

Scene 9: The front-end catches mistakes

A real compiler has to handle bad input well. This program has five mistakes: a missing semicolon, an unknown gate name, a gate using the same qubit twice, a division by zero, and an index out of range.

All five are reported at once, each with the exact line and column, a caret under the problem, and a suggestion, such as "did you mean h?".

If I fix one, the compiler reruns immediately and that error disappears.

I'll hand over to for the optimizer itself.

> **If asked "how does it keep going after an error?":**

Two recovery strategies. It skips to the next semicolon and continues. A missing semicolon at the end of a line is reported and then assumed, so the next line is still checked.

SPEAKER 3

Scene 10: Optimization 1: follow each qubit's wire

The first optimization is about where to look. Instead of checking lines of text, we follow each qubit's wire.

The obvious pairs go first: H twice, X twice, CX twice.

This one is more interesting. These two rotations on q1 are not next to each other: there's a CNOT between them. But that CNOT only uses q1 as its control, and a rotation of this kind cannot interfere with a control. So the first rotation slides past it, meets the second, and since they add up to zero, both disappear.

The rule for when two gates may slide past each other was checked by machine, for every pair of gates in every position.

Scene 11: ...and never when it is unsafe

Just as important is what the optimizer refuses to do. Dashed brackets mark gates it chose not to touch. Dotted brackets mark what stopped it.

On the left, two H gates on the same qubit look like a pair, but a measurement sits between them. Measuring in between changes the result, so they must stay. Two more H gates have an X between them, which doesn't commute with H, so they stay too.

On the right, the program takes two qubits as parameters, q0 and q1. They might actually be the same qubit, which is called aliasing. So the optimizer won't reorder anything across them.

Every refusal is explained. Missing an optimization is acceptable. Breaking a program is not.

Scene 12: Optimization 2: knowing a qubit's value

The second optimization is the main new idea. Remember the question from earlier: is a qubit definitely 0, definitely 1, or unknown? The compiler tracks that for every qubit, and then carries the answer into the classical code.

In this program, q0 is set to 1. So the CX that copies it always fires, and becomes a plain X. Now q1 is known to be 1, so measuring it always gives 1. The if-statement checks for 0, so its condition is always false. That block can never run, so it is deleted. The result is a smaller program with no branches left.

Here it is happening, one rule at a time: CX becomes X, the measurement becomes a constant, the comparison folds, the branch folds, the dead block is removed, the blocks merge, and the H pair cancels. Five gates and three blocks become two gates and one block, still proven equivalent.

Now the same program, but q0 starts in a 50/50 mixture. Nothing is known about q1, so the measurement is random and the branch might run. The optimizer still removes the H pair, but it keeps the CX and the branch, and it tells us why: the condition depends on a qubit in superposition.

For compiler people, this is sparse conditional constant propagation, branch folding, unreachable-code elimination and block merging, from the Dragon Book, with qubit values as the constants.

Over to for how we know all of this is correct.

> **If asked "is this safe with entangled qubits?":**

Yes. A qubit is only marked as known if it holds that value on every run and isn't linked to anything unknown. Entangled qubits are always "unknown".

SPEAKER 4

Scene 13: Every change is proven

How do we know every one of these rewrites is correct? We don't assume it. We check every one.

Each time the optimizer changes the program, a simulator runs the old and the new version and compares every possible measurement outcome. If anything differs, the change is rejected. This is called translation validation.

To show the checker really catches mistakes, we built a deliberately unsafe optimizer that matches gates by name, the way a simple text matcher would. In each case it does something that looks reasonable, but the output probabilities change, shown in red, and the checker rejects it. Ours is accepted every time.

This runs live, in this browser. It generates random programs, optimizes each one, and proves it equivalent. Our optimizer has zero counterexamples. The unsafe one, run on the same programs, gets caught hundreds of times.

Scene 14: How we are different

So how is our project different from what already exists? Here is the same small program after three kinds of optimizer.

An LLVM-style optimizer knows nothing about gates, so it keeps all five gates and all three blocks.

Gate rules alone, which is what circuit optimizers like Qiskit do, remove the two H gates. But they can't see that the branch can never run, so three gates and three blocks remain.

Our optimizer also knows q0 and q1 are 1, so it turns the CX into an X, proves the branch dead and deletes it. Two gates, one block, proven equivalent.

To be honest about prior work: cancelling gates is standard, and tracking qubit values has been done on circuits, for example in RPO at CGO 2021 and QCP at SAS 2023. What is new here is doing it directly on QIR, carrying it through real branches and classical code, and proving every single change.

> **If asked "isn't QCP more powerful?":**

For entangled qubits, yes, it tracks more. Our contribution is the layer and the integration, not a more precise analysis. A QCP-style analysis could be plugged into our framework as future work.

Scene 15: Results

Across 24 benchmark programs, the full optimizer removes 48 percent of all gates, twice what our Review 2 prototype managed, at 24 percent.

In each group, the light bar is the Review 2 approach, the middle bar adds following qubit wires, and the dark bar is the full optimizer. The adders are the clearest case: they barely shrink with gate rules alone, but shrink by about 83 to 84 percent once qubit values are tracked. That is the new part paying off.

And everything is checked: LLVM's own verifier accepts every output, the simulator proves every program it can enumerate equivalent, and all 69 automated tests pass.

> **If asked "why isn't the QFT example fully removed?":**

That needs a more advanced technique, called phase polynomials, which is future work. We report the result as it is.

SPEAKER 1, LIVE DEMO

Scene 16: Try it: any program

This is the same engine, open for any program. You can type a program or add gates with these buttons, switch individual optimizations on and off, replay every step, and see the results on a simulated noisy machine.

Would you like to give us a program to try?

SPEAKER 4, CLOSING

Scene 17: Limits and next steps

Finally, the limits, stated plainly. First, proof by simulation gets exponentially more expensive, so it stops at about 20 qubits; beyond that we report "not checked". Second, our analysis treats entangled qubits as unknown, so a more precise analysis would find more. Third, our noise figures come from a simulation, not from real hardware.

The next steps are smarter rotation merging, a richer qubit analysis, a version that plugs into LLVM's own pass manager, and running on real quantum hardware. We think the integration

of these ideas on QIR is strong enough for a short workshop paper.

Thank you. We're happy to take questions.

FOR THE VIVA

The appendix at the bottom of the presentation (**Show the write-up**) contains:

- the full technical write-up;
- every benchmark;
- the list of all tests;
- the references;
- prepared answers to likely questions.